

Orientações
Entrega documento: N/A Valor: N/A; Tema: Percurso em árvores (pré-ordem, pós-ordem, ordem).

1. Contar o total de nós da árvore

Contexto: Antes de gerar um relatório, o sistema precisa saber quantos registros estão armazenados na árvore.

Classe: `ArvoreBinaria` — adicione o método `int contarNos()`

Percurso: Qualquer (a contagem não depende da ordem de visita)

Saída esperada — valores { 10, 5, 15, 3, 7, 12, 20 }

`Total de nos: 7`

2. Imprimir apenas os nós folha

Contexto: Em um catálogo de categorias, os nós folha são os itens finais, sem subcategorias.

Classe: `ArvoreBinaria` — adicione o método `void imprimirFolhas()`

Percurso: **Pré-ordem** — processa o nó antes de descer; ao encontrar folha (`esquerdo == null && direito == null`), imprime.

Saída esperada — valores { 4, 2, 6, 1, 3, 5, 7 }

`Folhas: 1 3 5 7`

3. Verificar se um valor existe na árvore

Contexto: Antes de inserir um novo produto, o sistema verifica se o código já existe.

Classe: `ArvoreBinaria` — adicione o método `boolean contem(int info)`

Dica: Aproveite a propriedade da ABP: a cada nó, decida esquerda ou direita sem percorrer os dois lados.

Saída esperada — valores { 15, 8, 22, 4, 11 }

`contem(11): true`

`contem(99): false`

4. Relatório de alunos em ordem crescente de RA

Contexto: RAs inseridos em ordem aleatória. O coordenador quer o relatório em ordem crescente sem usar nenhum algoritmo de ordenação extra.

Classe: `RelatorioAlunos` — herda de `EmOrdem`. Sobrescreva `percorrer()` para imprimir RA: 1023 por linha.

Percurso: **Em Ordem** — Esq → Raiz → Dir produz saída crescente automaticamente na ABP.

Saída esperada — valores { 1045, 1023, 1078, 1001, 1033, 1060, 1090 }

```
RA: 1001
RA: 1023
RA: 1033
RA: 1045
RA: 1060
RA: 1078
RA: 1090
```

Justifique num comentário por que *Em Ordem* garante essa saída sem ordenação extra.

5. Serializar a árvore para reconstrução (backup)

Contexto: O sistema precisa salvar a estrutura da árvore num array para reconstruí-la identicamente depois.

Classe: `ArvoreBinaria` — adicione o método `int[] serializar()`

Percurso: **Pré-Ordem** — Raiz → Esq → Dir. Reinserindo os valores nessa ordem, a árvore é reconstruída com a mesma estrutura.

Depois de serializar, crie uma nova `ArvoreBinaria`, insira os valores do array na ordem salva e confirme que o em-ordem das duas árvores é idêntico.

Saída esperada — valores { 4, 2, 6, 1, 3, 5, 7 }

```
Serializado (pre-ordem): 4 2 1 3 6 5 7
Em-ordem original:      1 2 3 4 5 6 7
Em-ordem reconstruida:  1 2 3 4 5 6 7
```

Justifique num comentário por que *pré-ordem* é o único percurso que preserva a estrutura.

6. Calcular a soma de todos os valores da árvore

Contexto: Um auditor quer a soma de todos os códigos armazenados para conferência de integridade.

Classe: `ArvoreBinaria` — adicione o método `int somarNos()`

Percurso: Pós-Ordem — some os filhos antes de somar o nó atual; o resultado de cada subárvore sobe para o pai.

Saída esperada — valores { 10, 5, 15, 3, 7, 12, 20 }

Soma total: 72

Justifique num comentário por que pós-ordem faz sentido aqui.

7. Calcular a altura da árvore

Contexto: Para verificar se a árvore está equilibrada, o sistema calcula o número de níveis (altura).

Classe: `ArvoreBinaria` — adicione o método `int altura()`

Regra: Árvore vazia → 0. Nó folha → 1. Demais → `1 + Math.max(altEsq, altDir)`.

Percurso: Pós-Ordem — calcule a altura dos filhos antes de calcular a do nó atual.

Teste com duas árvores: { 4, 2, 6, 1, 3, 5, 7 } (balanceada) e { 1, 2, 3, 4, 5 } (degenerada, inserida em ordem crescente).

Saída esperada

Altura balanceada: 3

Altura degenerada: 5